

Chapter 2

Data Transmission

Cambridge IGCSE Computer Science 2210 — Revision Notes

created & compiled by Afrasiab

In this chapter: data packets & packet switching · transmission modes & USB · error detection (parity, checksum, echo check, check digits, ARQ) · encryption (symmetric & asymmetric)

Data is constantly moved between devices – next door or across the world. Every time, three things matter: how it's sent, how errors are caught, and how it's kept safe from prying eyes. That's this whole chapter.

2.1 Types & Methods of Data Transmission

2.1.1 Data Packets

Data packet (datagram) – a small part of a message/data that is transmitted over a network; after transmission all the packets are reassembled to form the original data.

Data sent over long distances is broken into **packets**, usually around 64 KiB each. Splitting it up means each packet can travel a different route – handy if one path is busy or down. Downside: the packets must be reassembled correctly at the other end.

Packet structure

Every packet = Header + Payload + Trailer

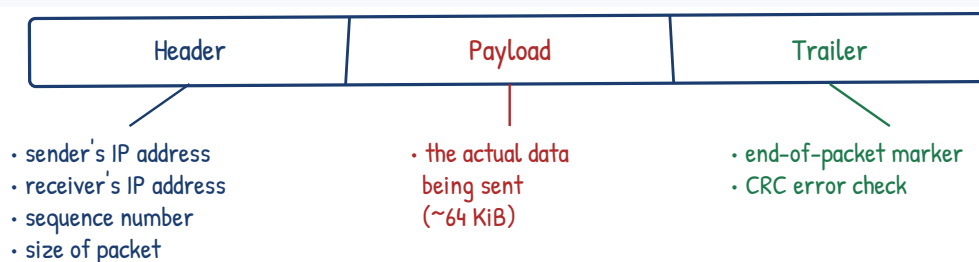


Fig 2.1 — packet structure

Cyclic Redundancy Check (CRC) – sender adds up all 1-bits in the payload & stores the total (as hex) in the trailer; receiver redoes the same sum and compares. Match = no error.

exam tip → Header = where it's going / came from + order. Trailer = where it ends + is it error-free. Don't swap these!

Packet switching

Packet switching – a message is broken into packets which travel independently and are reassembled at the destination using the header info.

- Each stage has a **node** containing a **router**, which reads the header and picks the shortest path currently available
- (not necessarily the shortest possible – parts of the route might be busy or broken).

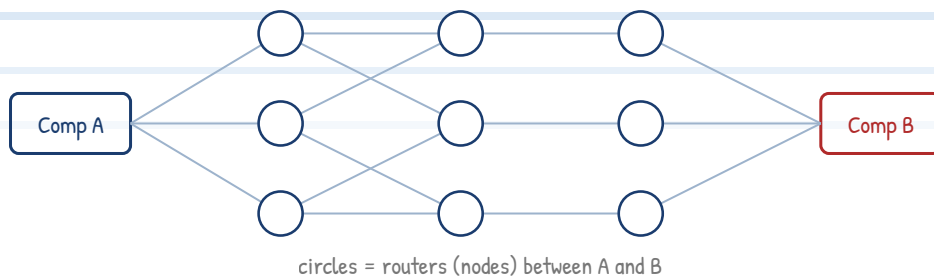


Fig 2.2 – possible packet-switched routes

Benefits	Drawbacks
no need to tie up one line	packets can get lost & need re-sending
can re-route round busy/faulty lines	worse for real-time streaming (e.g. live sport)
easy to expand usage	delay while packets are re-ordered
high transmission rate possible	packets may arrive out of order

note → "Hopping" (not on syllabus): each packet gets a hop number that drops by 1 every router it passes; hits zero → deleted, so lost packets don't clog the network forever.

Q-try these:

1. State two benefits of packet switching. [2]
2. Explain why packets can arrive out of order, and how the receiver deals with it. [4]
3. A live match is streamed using packet switching – explain why the picture might freeze. [3]

homework:

1. Define "data packet" and list what's in the header, payload & trailer.

2. Explain how a CRC works.

3. Explain packet switching & the role of a router.

quick revision – 2.1.1

- packet = header + payload + trailer

- header: IPs, sequence no., size

- trailer: end marker + CRC

- router picks shortest available path

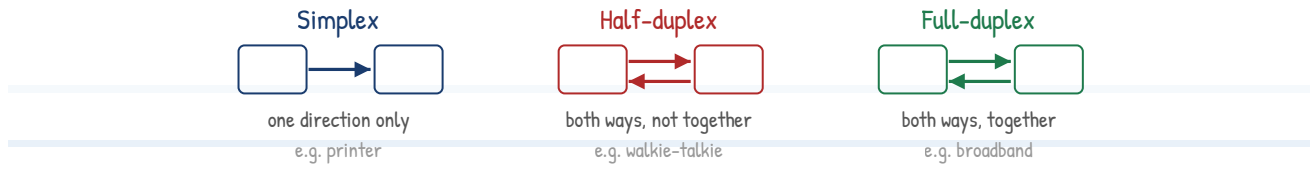
- packets reassembled using sequence no.

- mistake to avoid: packets always arrive in order – false!

2.1.2 Direction & Method of Transmission

Three things matter: direction, method (bits at once), and synchronisation – usually agreed by a communication protocol.

Simplex / half-duplex / full-duplex



Simplex – one direction only. **Half-duplex** – both directions, not at the same time. **Full-duplex** – both directions, at the same time.

Serial & parallel transmission

Serial – data sent ONE BIT at a time down ONE wire.

Parallel – SEVERAL bits (usually a byte) sent down SEVERAL wires at once.

Serial	Parallel
less interference (fewer wires)	faster than serial
reliable over long distances	only good over short distances
no skewing – stays synced	bits can arrive unsynced (skewed)
good for small amounts of data (e.g. USB)	good when speed matters
cheaper – less hardware	more expensive – more hardware

exam tip → "Skewing" = bits arriving out of sync. It's a parallel problem only, not serial.

Q-try these:

1. Explain serial, half-duplex data transmission. [2]
2. Name the transmission type: "16 bits at a time, both directions at once." [1]
3. Why is parallel transmission unsuitable over long distances? [2]

2.1.3 Universal Serial Bus (USB)

USB is a form of **serial** transmission – the most common I/O port today. Supports both half & full-duplex. 4 wires: 2 for power (+5V, ground), 2 for data. Plug a device in → computer detects the voltage change → loads the right driver automatically (or asks you to download one).

Benefits	Drawbacks
auto-detect & auto driver install	standard cable max 5 m (need hubs beyond)
fits only one way – no wrong connections	very old (V1) devices may not suit newest PCs
industry standard, well supported	even USB-C is slower than Ethernet
no external power needed (+5V from cable)	—
auto re-transmits if errors found	—
backward compatible	—

USB-C: 24-pin, fits either way round, up to 100W power & 10 Gbps (4K video capable). Backward compatible with an adaptor.

homework:

1. Define simplex, half-duplex, full-duplex with one example each.
2. Compare serial vs parallel (3 points).
3. State 3 benefits + 2 drawbacks of USB.

quick revision – 2.1.2 / 2.1.3

- | | |
|--|--|
| • simplex = 1 way; half-duplex = both, not together; | • parallel = many wires, short distance, risk of skewing |
| full = both, together | • USB = serial; half & full duplex; max 5 m standard |
| • serial = 1 wire 1 bit, long distance | • USB-C = 24-pin, reversible, up to 10 Gbps / 100 W |

2.2 Methods of Error Detection

Data can get corrupted, lost, or gained due to: interference, packet-switching problems, or skewing. Computers can't "guess" like humans do – even small corruption makes text meaningless to them. Three post-transmission checks: **parity**, **checksum**, **echo check**. (Check digits catch entry errors instead – see 2.2.3.)

2.2.2 Parity, Checksum & Echo Check

Parity check

Parity check – checks data using EVEN parity (even number of 1-bits) or ODD parity (odd number). Sender & receiver agree which, beforehand.

One bit (usually leftmost) is the **parity bit**.

parity bit	b2	b3	b4	b5	b6	b7
?	1	1	0	1	1	0 0

4 ones already → even parity bit = 0 | odd parity bit = 1

worked example → sent (even parity): 0 1 0 1 1 1 0 0 (four 1s, correct) received: 0 1 0 0 1 1 0 0 (three 1s – now odd) → **error detected!**

limitation → if two bits flip, parity can stay the same → error goes undetected.

Parity blocks – finding the exact bit

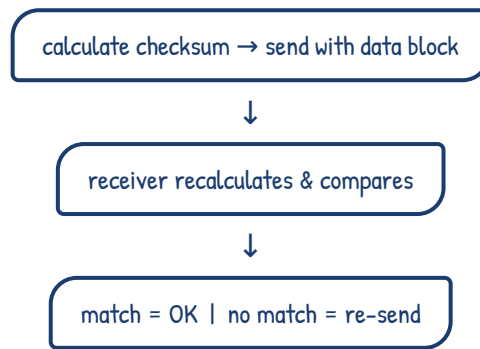
Send a block of bytes; check parity horizontally (each byte) and vertically (each bit position → parity byte). Where a bad row and bad column cross = the exact faulty bit.

	parity	b2	b3	b4	b5	b6	b7	b8
byte 1	1	1	1	1	0	1	1	0
byte 8	0	0	0	1	1	0	1	0
parity byte	1	1	0	1	0	0	0	1

byte 8's row and column 5 both show bad parity → that exact bit is wrong → flip it to fix

Checksum

Checksum – a value calculated from a data block using an agreed algorithm, sent along with the block, then recalculated & compared at the receiver.



Echo check

Echo check – receiver sends the data straight back; sender compares it with the original.

limitation → if they don't match, you can't tell if the error happened going out or coming back.

Q-try these:

1. Byte 1101101 uses even parity – find the parity bit. [1]
2. Why can parity checking fail to catch an error? [2]
3. Describe how a parity block finds the exact faulty bit. [4]

homework:

1. Define parity check, checksum, echo check in your own words.
2. Check: does 1011011 have odd parity? Show working.
3. State one weakness of each method above.

2.2.3 Check Digits

Check digit – the final digit of a code, calculated from the other digits, used to catch DATA ENTRY errors (not transmission errors).

exam tip → Check digit = data entry error. Parity/checksum/echo/ARQ = data transmission error. Don't mix these up!

Catches: wrong digit · transposed digits (swap) · missing/extra digit · phonetic slip (13 vs 30).

Method 1 – ISBN-13

worked example → Number: 9 7 8 0 3 4 0 9 8 3 8 2

1) odd positions: $9+8+3+0+8+8 = 36$

2) even positions $\times 3$: $3 \times (7+0+4+9+3+2) = 75$

3) $(36+75)/10 = 111/10 = \text{remainder } 1$

4) check digit = $10-1 = 9 \rightarrow \text{final: } 9\ 7\ 8\ 0\ 3\ 4\ 0\ 9\ 8\ 3\ 8\ 2\ 9$

Method 2 – Modulo-11

worked example → Number: 4 1 5 6 7 1 0 weights: 8 7 6 5 4 3 2

1) $(8 \times 4) + (7 \times 1) + (6 \times 5) + (5 \times 6) + (4 \times 7) + (3 \times 1) + (2 \times 0) = 130$

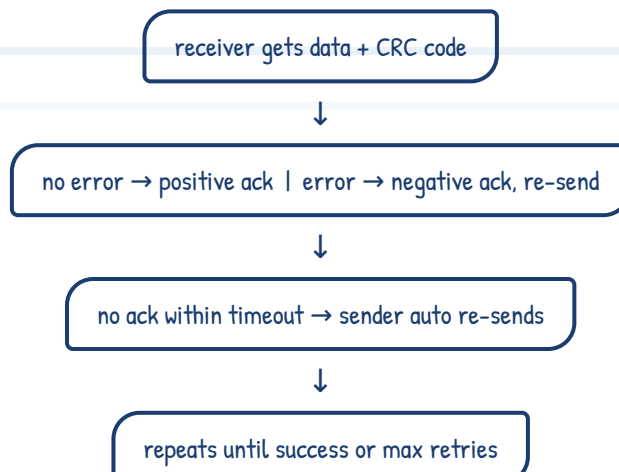
2) $130 \div 11 = \text{remainder } 9$

3) check digit = $11-9 = 2$ (if remainder=10, digit is "X") → final: 4 1 5 6 7 1 0 2

memory trick → ISBN-13 → $\div 10$. Modulo-11 → $\div 11$ (it's in the name!)

2.2.4 Automatic Repeat Request (ARQ)

ARQ – uses acknowledgement + timeout to automatically request re-sending if no positive response arrives in time.



used by mobile phone networks to guarantee data integrity

Q—try these:

1. Calculate the ISBN-13 check digit for: 978151045759 [4]
2. Find the check digit for 21311100428 using Modulo-11. [4]
3. State two error types a check digit can detect. [2]

homework:

1. Define acknowledgement & timeout as used in ARQ.
2. Explain what happens step by step when ARQ detects an error.
3. Calculate the ISBN-13 check digit for: 970150449512

quick revision – 2.2

- parity: even/odd 1-bit count; fails on 2-bit errors
- check digit = ENTRY error, not transmission
- parity block: row × column locates exact bit
- ISBN-13: ÷10; Modulo-11: ÷11
- checksum: value sent + block, recompared
- ARQ = acknowledgement + timeout + auto re-send
- echo check: sent back, can't tell which leg failed

2.3 Symmetric & Asymmetric Encryption

Data on a public network risks interception by an **eavesdropper** (a hacker). Encryption doesn't stop interception – it makes intercepted data meaningless without the right key. Vital for bank details, medical records, legal docs.

Plaintext – original data before encryption. **Ciphertext** – scrambled data after encryption.



Symmetric encryption

Symmetric encryption – the SAME key encrypts and decrypts.

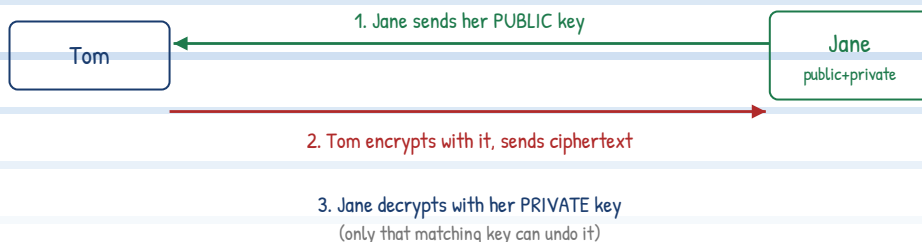
worked example → key (shifts): 4 2 9 1 3 6 2 8 5 6 → COMPUTERSC → GQVQXZGZXI (decrypt = shift back by same key)

Real systems use 256-bit keys (2^{256} combinations) – small keys can be cracked in seconds.

weakness → the one key must be sent somehow (email/text) – it could be intercepted. This is the key distribution problem.

Asymmetric encryption

Asymmetric encryption – uses a mathematically-linked PUBLIC key (known to everyone) + PRIVATE key (known only to the owner). Both needed to encrypt/decrypt.



Symmetric

one key both ways

faster, but key must be shared (risk!)

risk: key distribution problem

Asymmetric

public + private key pair

slower, but private key never travels

public key safe to share with anyone

exam tip → Why is asymmetric more secure? Because the private key never gets transmitted – it stays on the owner's computer.

memory trick → public key = your house address (anyone can know it); private key = your house key (only you can open the door).

Q–try these:

1. Define plaintext and ciphertext. [2]
2. Why is key distribution a weakness of symmetric encryption? [2]
3. Describe how Tom sends Jane a file securely using asymmetric encryption. [4]

homework:

1. Define symmetric & asymmetric encryption.
2. Draw & explain how Tom sends Jane an encrypted file.
3. State one advantage of symmetric vs asymmetric encryption.

quick revision – 2.3

- plaintext → algorithm → ciphertext
- symmetric = same key; weak point = key distribution
- asymmetric = public (shared) + private (secret)
- public key encrypts; private key decrypts
- private key never transmitted → more secure

Chapter Revision

Keyword sheet

Data packet – small part of a message, reassembled after transmission

Header – source IP, dest IP, sequence no., size

Trailer – end marker + CRC

CRC – adds up 1-bits, compares at receiver

Payload – the actual data in a packet

Router / node – forwards packets across a network

Packet switching – message split into independently-sent packets

Simplex – one direction only

Half-duplex – both directions, not together

Full-duplex – both directions, together

Serial – 1 bit, 1 wire

Parallel – many bits, many wires

Skewed data – bits arrive unsynchronised

USB – serial; half & full duplex; 5 m max standard

Parity check – even/odd 1-bit count

Checksum – calculated value sent with a block

Echo check – data echoed back for comparison

Check digit – catches data ENTRY errors

ARQ – acknowledgement + timeout + auto re-send

Eavesdropper – hacker intercepting data

Plaintext / Ciphertext – before / after encryption

Symmetric encryption – one key, both ways

Asymmetric encryption – public + private key pair

Top examiner mistakes

1. Check digit = data entry error, not transmission.
2. Skewing is a parallel-only problem.
3. Packet switching does not guarantee order – sequence number fixes this.
4. Parity fails if exactly two bits change – don't say it "always" works.
5. Private key is never transmitted in asymmetric encryption.
6. USB is serial, not parallel.
7. ARQ needs both acknowledgement and timeout.

Last-minute checklist

- ☐ I can draw & label packet structure
- ☐ I can explain packet switching & routers
- ☐ I can define simplex / half-duplex / full-duplex with examples

- ☐ I can compare serial vs parallel (3+ points)
- ☐ I know USB is serial, 3 benefits + 2 drawbacks
- ☐ I can calculate a parity bit & locate an error in a parity block
- ☐ I can explain checksum & echo check + their weaknesses
- ☐ I can calculate an ISBN-13 check digit
- ☐ I can calculate a Modulo-11 check digit
- ☐ I can explain ARQ (acknowledgement + timeout)
- ☐ I can define plaintext/ciphertext/symmetric/asymmetric
- ☐ I can explain why asymmetric encryption is more secure

— end of chapter 2 —

created & compiled by Afrasiab